

Personalized Learning AI — Technical Stack

Recommended stack for OpenTutor: an open-source adaptive learning platform with trainable AI components and a production-style web application.

Layer	Recommended technology	Purpose
Frontend	Next.js + TypeScript	Web application and learner dashboard
UI	Tailwind CSS + shadcn/ui	Accessible, reusable interface
Charts	Recharts	Mastery, progress and evaluation visualizations
Backend	Python + FastAPI	REST API, inference orchestration
Validation	Pydantic	Typed request/response contracts
Database	PostgreSQL	Users, courses, attempts, learner state
ORM	SQLAlchemy	Database access and migrations
Cache/queue	Redis	Caching and background jobs
Workers	Celery or RQ	Long-running model/data tasks
ML	PyTorch	Model training and experimentation
NLP	Hugging Face Transformers + Datasets	Open models, tokenization, datasets
Fine-tuning	PEFT + TRL	LoRA/QLoRA and supervised fine-tuning
Embeddings	Sentence Transformers	Semantic retrieval/feature generation
Vector DB	Qdrant	Optional content retrieval and concept search
Experiment tracking	MLflow	Runs, metrics, artifacts
Evaluation	scikit-learn + custom evaluators	Classification/recommendation metrics
Data	Pandas + PyArrow	Dataset preparation and storage
Model serving	vLLM or Transformers	Efficient inference; start with Transformers
Containers	Docker + Compose	Reproducible local/deployment environment
CI	GitHub Actions	Tests, lint, build checks
Testing	pytest + Playwright	Backend/ML + end-to-end web tests
Formatting	Ruff + Black	Python linting/formatting
Frontend quality	ESLint + Prettier	TypeScript/React quality
Model registry	Hugging Face Hub	Publish model and dataset
Demo	Hugging Face Spaces or deployed web app	Public AI demonstration
Frontend hosting	Vercel	Simple Next.js deployment
API hosting	GPU cloud / VM	Inference and background processing
Licensing	Apache-2.0 for code; dataset/model licenses chosen separately	Open-source distribution with clear artifact licensing

Core AI architecture

Learner interactions → misconception model → learner state / mastery model → prerequisite graph → recommendation policy → next activity → new interaction → updated learner state.

Model strategy

Do not train a large language model from scratch. Start from a compact open model and fine-tune it with LoRA/QLoRA. Keep the personalized learning/recommendation model separate from the language model so

the research contribution is measurable.

Suggested first model stack

- 1) Compact open LLM for misconception detection and educational response generation.
- 2) A tabular/sequence model for mastery and next-activity prediction.
- 3) A prerequisite/concept graph for curriculum structure.
- 4) Optional RAG for grounding explanations in course material.